

Nama: Muhammad Akmal Dwiansyah Putra

Kelas / Absen: TI_1G / 20

NIM: 254107020110

Praktikum Jobsheet 12

Praktikum 1:

Mahasiswa20.java:

```
package Jobsheet12;
```

```
public class Mahasiswa20 {
```

```
    String nim;
```

```
    String nama;
```

```
    String kelas;
```

```
    double ipk;
```

```
    Mahasiswa20(String nim, String nama, String kelas, double ipk){
```

```
        this.nim = nim;
```

```
        this.nama = nama;
```

```
        this.kelas = kelas;
```

```
        this.ipk = ipk;
```

```
    }
```

```
    void tampil(){
```

```
        System.out.println("NIM : " + nim);
```

```
        System.out.println("Nama : " + nama);
```

```
        System.out.println("Kelas : " + kelas);
```

```
        System.out.println("IPK : " + ipk);
```

```
        System.out.println("=====");
```

```
    }
```

```
}
```

Node20.java:

```
package Jobsheet12;

public class Node20 {
    Mahasiswa20 data;
    Node20 prev;
    Node20 next;

    Node20(Mahasiswa20 data){
        this.data = data;
        prev = null;
        next = null;
    }
}
```

DoubleLinkedList20.java:

```
package Jobsheet12;

public class DoubleLinkedList20 {
    Node20 head;
    Node20 tail;

    DoubleLinkedList20(){
        head = null;
        tail = null;
    }

    boolean isEmpty(){
        return head == null;
    }
}
```

```
void addFirst(Mahasiswa20 data){
    Node20 newNode = new Node20(data);
    if (isEmpty()) {
        head = tail = newNode;
    }else{
        newNode.next = head;
        head.prev = newNode;
        head = newNode;
    }
}
```

```
void addLast(Mahasiswa20 data){
    Node20 newNode = new Node20(data);
    if (isEmpty()) {
        head = tail = newNode;
    }else{
        tail.next = newNode;
        newNode.prev = tail;
        tail = newNode;
    }
}
```

```
void add(Mahasiswa20 data, int index){

}
```

```
void print(){
    if (isEmpty()) {
        System.out.println("Linked list masih kosong");
    }
}
```

```
    return;  
}
```

```
Node20 current = head;  
while (current != null) {  
    current.data.tampil();  
    current = current.next;  
}  
}
```

```
void removeFirst(){  
  
}
```

```
void removeLast(){  
  
}
```

```
void insertAfter(String keyNIM, Mahasiswa20 data){  
    Node20 current = head;  
    while (current != null && !current.data.nim.equals(keyNIM)) {  
        current = current.next;  
    }  
    if (current == null) {  
        System.out.println("Data dengan NIM " + keyNIM + " tidak ditemukan!");  
        return;  
    }  
}
```

```
Node20 newNode = new Node20(data);
```

```

    if (current == tail) {
        newNode.prev = current;
        current.next = newNode;
        tail = newNode;
    } else {
        newNode.prev = current;
        newNode.next = current.next;
        newNode.next.prev = newNode;
        current.next = newNode;
    }
    System.out.println("Data berhasil disisipkan setelah NIM " + keyNIM);
}
}

```

DoubleLinkedListMain20.java:

```

package Jobsheet12;

import java.util.Scanner;

public class DoubleLinkedListMain20 {
    static Mahasiswa20 inputMahasiswa20(Scanner scan){
        System.out.print("NIM : ");
        String nim = scan.nextLine();
        System.out.print("Nama : ");
        String nama = scan.nextLine();
        System.out.print("Kelas : ");
        String kelas = scan.nextLine();
        System.out.print("IPK : ");
        double ipk = scan.nextDouble();
        scan.nextLine();
    }
}

```

```

        return new Mahasiswa20(nim, nama, kelas, ipk);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        DoubleLinkedList20 list = new DoubleLinkedList20();
        int pilih;

        do {
            System.out.println("\n=== MENU DOUBLE LINKED LIST ===");
            System.out.println("1. Tambah data diawal");
            System.out.println("2. Tambah data diakhir");
            System.out.println("3. Sisipkan data setelah NIM");
            System.out.println("4. Hapus data diawal");
            System.out.println("5. Hapus data diakhir");
            System.out.println("6. Tampilkan data");
            System.out.println("0. Keluar");
            System.out.print("Pilih Menu: ");

            pilih = sc.nextInt();
            sc.nextLine();

            switch (pilih) {
                case 1:
                    Mahasiswa20 mhsAwal = inputMahasiswa20(sc);
                    list.addFirst(mhsAwal);
                    break;
                case 2:
                    Mahasiswa20 mhsAkhir = inputMahasiswa20(sc);

```

```

        list.addLast(mhsAkhir);

        break;
    case 3:
        System.out.print("Masukkan NIM yang dicari: ");
        String keyNIM = sc.nextLine();
        System.out.println("Input data Mahasiswa: ");
        Mahasiswa20 mhsTengah = inputMahasiswa20(sc);
        list.insertAfter(keyNIM, mhsTengah);
        break;
    case 4:
        list.removeFirst();
        break;
    case 5:
        list.removeLast();
        break;
    case 6:
        list.print();
        break;
    case 0:
        System.out.println("Program Berhenti");
        break;
    default:
        System.out.println("Input Tidak Valid");
    }
} while (pilih != 0);
sc.close();
}
}

```

Hasil:

```
=== MENU DOUBLE LINKED LIST ===
```

1. Tambah data diawal
2. Tambah data diakhir
3. Sisipkan data setelah NIM
4. Hapus data diawal
5. Hapus data diakhir
6. Tampilkan data
0. Keluar

Pilih Menu: 2

NIM : 123005

Nama : Harry

Kelas : 1A

IPK : 3.76

```
=== MENU DOUBLE LINKED LIST ===
```

1. Tambah data diawal
2. Tambah data diakhir
3. Sisipkan data setelah NIM
4. Hapus data diawal
5. Hapus data diakhir
6. Tampilkan data
0. Keluar

Pilih Menu: 3

Masukkan NIM yang dicari: 123005

Input data Mahasiswa:

NIM : 123010

Nama : Potter

Kelas : 1B

IPK : 3.55

Data berhasil disisipkan setelah NIM 123005

=== MENU DOUBLE LINKED LIST ===

1. Tambah data diawal
2. Tambah data diakhir
3. Sisipkan data setelah NIM
4. Hapus data diawal
5. Hapus data diakhir
6. Tampilkan data
0. Keluar

Pilih Menu: 6

NIM : 123005

Nama : Harry

Kelas : 1A

IPK : 3.76

=====

NIM : 123010

Nama : Potter

Kelas : 1B

IPK : 3.55

=====

=== MENU DOUBLE LINKED LIST ===

1. Tambah data diawal
2. Tambah data diakhir
3. Sisipkan data setelah NIM
4. Hapus data diawal
5. Hapus data diakhir
6. Tampilkan data
0. Keluar

Pilih Menu:

12.2.3 Pertanyaan

1. Jelaskan perbedaan struktur dan mekanisme traversal antara Single Linked List dan Double Linked List!

Jawaban: Perbedaan dari Single dan Double Linked List adalah Double Linked List memiliki node baru yaitu prev atau sebelum. Ini dapat digunakan oleh DLL untuk berpindah ke arah sebelumnya / belakang, yang tidak dapat dilakukan oleh SLL.

2. Perhatikan class Node, di dalamnya terdapat atribut next dan prev. Jelaskan fungsi masing-masing atribut tersebut pada proses traversal dan manipulasi node!

Jawaban: Kegunaan dari dari next dan prev dalam proses traversal dan manipulasi node rata-rata sama, yang membedakan keduanya adalah bagaimana cara node tersebut berinteraksi pada proses DLL, next akan melakukan proses traversal dan manipulasi di bagian kanan atau setelah dari data sekarang, sedangkan prev akan melakukan proses traversal dan manipulasi dibagian kiri atau sebelum dari data saat ini.

3. Perhatikan konstruktor pada class DoubleLinkedList. Jelaskan fungsi konstruktor tersebut terhadap kondisi awal linked list!

Jawaban: Fungsi dari memberikan nilai null pada head dan tail adalah untuk memberikan nilai kepada isEmpty(), karena masih belum di beri input maka isEmpty() akan bernilai true dan hal ini akan membuat program tidak menjalankan error

4. Perhatikan potongan kode berikut:

```
if (isEmpty()) {  
    head = tail = newNode;  
}
```

Mengapa head dan tail harus menunjuk node yang sama ketika linked list masih kosong?

Jawaban: Karena pada awal memasukan data kepada linked list yang masih kosong, data yang dimasukkan tersebut termasuk head dan juga tail dari linked list. Tail atau head akan berbeda jika ditambahkan data baru pada linked list tersebut.

5. Modifikasi method print() agar menampilkan pesan "Linked List masih kosong" ketika tidak terdapat data pada linked list!

Jawaban:

```
if (isEmpty()) {  
    System.out.println("Linked list masih kosong");  
    return;  
}
```

6. Modifikasi kode program dengan menambahkan method `printReverse()` untuk menampilkan seluruh data pada Double Linked List secara terbalik, dimulai dari node tail menuju head!

Jawaban:

```
void printReverse(){
    if (isEmpty()) {
        System.out.println("Linked list masih kosong");
        return;
    }

    Node20 current = tail;
    while (current != null) {
        current.data.tampil();
        current = current.prev;
    }
}
```

Praktikum 2:

DoubleLinkedList20.java:

```
void removeFirst(){
    if (isEmpty()) {
        System.out.println("Linked list masih kosong");
        return;
    }

    if (head == tail) {
        head = tail = null;
    }else{
        head = head.next;
        head.prev = null;
    }
}
```

```
}
```

```
void removeLast() {  
    if (isEmpty()) {  
        System.out.println("Linked list masih kosong");  
        return;  
    }  
  
    if (head == tail) {  
        head = tail = null;  
    } else {  
        tail = tail.prev;  
        tail.next = null;  
    }  
}
```

Hasil:

NIM : 123005

Nama : Harry

Kelas : 1A

IPK : 3.76

=====

NIM : 123010

Nama : Potter

Kelas : 1B

IPK : 3.55

=====

=== MENU DOUBLE LINKED LIST ===

1. Tambah data diawal
2. Tambah data diakhir
3. Sisipkan data setelah NIM
4. Hapus data diawal
5. Hapus data diakhir
6. Tampilkan data
7. Tampilkan dari belakang
0. Keluar

Pilih Menu: 5

```

=== MENU DOUBLE LINKED LIST ===
1. Tambah data diawal
2. Tambah data diakhir
3. Sisipkan data setelah NIM
4. Hapus data diawal
5. Hapus data diakhir
6. Tampilkan data
7. Tampilkan dari belakang
0. Keluar
Pilih Menu: 6
NIM : 123005
Nama : Harry
Kelas : 1A
IPK : 3.76
=====

```

12.3.3 Pertanyaan Percobaan

1. Perhatikan potongan kode berikut pada method `removeFirst()`:
`head = head.next;`
`head.prev = null;`
 Jelaskan fungsi masing-masing statement tersebut pada proses penghapusan node!

Jawaban: Pada method tersebut proses dari penghapusan node adalah head pindah ke kanan atau next, lalu prev atau kiri dari head baru ini akan diubah menjadi null.

2. Modifikasi method `removeFirst()` dan `removeLast()` agar program menampilkan data yang berhasil dihapus!

Jawaban:

```

void removeFirst(){
    if (isEmpty()) {
        System.out.println("Linked list masih kosong");
        return;
    }

    if (head == tail) {
        head = tail = null;
    }
}

```

```
    }else{  
        head = head.next;  
        head.prev.data.tampil();  
        head.prev = null;  
    }  
}
```

```
void removeLast(){  
    if (isEmpty()) {  
        System.out.println("Linked list masih kosong");  
        return;  
    }  
}
```

```
if (head == tail) {  
    head = tail = null;  
}else{  
    tail = tail.prev;  
    tail.next.data.tampil();  
    tail.next = null;  
}  
}
```